Autonomous Codebase Architect

I. Project Overview

The **Autonomous Codebase Architect** is an advanced AI agent designed to reverse-engineer entire codebases and allow users to chat directly with the architecture. Instead of relying on a standard chat interface, this system autonomously clones repositories, extracts Abstract Syntax Trees (AST), maps internal dependencies, and utilizes a ReAct (Reasoning and Acting) loop to synthesize comprehensive onboarding guides.

The inference engine is heavily optimized, running the `Qwen/Qwen2.5-Coder-7B-Instruct-AWQ` model natively on an AMD Radeon Cloud GPU via an OpenAI-compatible vLLM server.

ere is the expanded, highly detailed version of Sections II and III. This dives deep into the technical rigor of your LangGraph state machine and custom AST extraction tools, proving to the judges exactly how sophisticated your pipeline is.

II. System Architecture & Workflow

The Autonomous Codebase Architect is orchestrated by a deterministic, multi-agent state machine built on `langgraph`. Rather than a standard linear prompt chain, the architecture features cyclical routing, deep state memory, and autonomous self-correction.

1. The Global Agent State

The entire pipeline is governed by a heavily typed `AgentState` dictionary that serves as the system's memory. As the agent traverses the graph, it progressively enriches this state with:

- **Raw Data:** Local repository paths, valid file lists, and original README contents.
- **Structured Intelligence:** Abstract Syntax Trees (AST), directed dependency graphs, and calculated graph centrality metrics (critical nodes).
- **Conversational Memory:** Extracted subsystem summaries, RAG context, and ReAct observations.

2. The ReAct (Reasoning & Acting) Orchestrator

At the center of the graph is the `Dynamic Planner`, acting as the system's brain.

- **Observation Loop:** Before deciding on a move, the planner analyzes past observations and the current user request to generate a step-by-step strategy.

- **Dynamic Routing:** The planner outputs an explicit action word—either routing the graph to `RETRIEVE` specific code chunks from the vector database, or moving to `GENERATE` the final architectural summary.
- **Safety Overrides:** To prevent infinite execution loops during live inference, the planner features a hard-coded circuit breaker that force-quits if the node count exceeds 8 steps. Furthermore, if context is successfully retrieved, an override forces the state forward to the generation phase.

3. Contextual UI Injection

To maintain human-like conversational memory across graph executions, the Streamlit frontend (`app.py`) utilizes LangGraph's `MemorySaver`tied to a unique session UUID. The UI intercepts the last four messages of the chat history, truncates them to 800 characters to protect the LLM context window, and dynamically injects them into the payload before triggering the graph.

4. Autonomous QA & Reflection

Instead of delivering the first draft directly to the user, the generated guide is routed to a `QA Reviewer` node.

- **Strict Rubric:** For general architecture requests, the reviewer parses the markdown draft to ensure four critical sections are explicitly present: "The 10,000-Foot View", "The Heavy Lifters", "The Execution Flow", and "Where to Start".
- **Self-Correction:** If any section is missing, the reviewer fails the draft, generates specific critique feedback (e.g., "Missing: The Execution Flow"), and routes the execution back to the Generator node to rewrite it.
- **Stall Prevention:** The reflection loop includes a `revision_count` tracker that accepts the best available draft after 3 failed attempts, guaranteeing a final output.

III. Core Tools & Capabilities

To reverse-engineer an entire codebase autonomously, the agent relies on a specialized suite of extraction tools that blend traditional static analysis with semantic AI embeddings.

1. High-Speed Repository Ingestion

The extraction pipeline begins with a highly optimized Git clone sequence.

- **Shallow Cloning:** The `clone_repo` tool utilizes `git.Repo.clone_from` with a `depth=1` flag to drastically reduce download times and disk space requirements.

- **Noise Filtration:** The `explore_files` function aggressively filters the cloned directory using `rglob`, automatically ignoring build artifacts, virtual environments (e.g., `node_modules, venv`), and irrelevant files, isolating only exact matches from a whitelist of allowed code extensions.

2. Dynamic AST Extraction (Tree-Sitter)

Rather than relying on the LLM to blindly read raw text, the system uses the `tree-sitter` library to syntactically parse the code.

- **Dynamic Language Registry:** The tool implements a failsafe `try/except` registry that loads grammars (Python, JavaScript, C++, Rust, etc.) on the fly. If a grammar is missing from the environment, the tool skips it gracefully without crashing the pipeline.
- **Universal Node Mapping:** The parser normalizes syntax trees across different languages—mapping language-specific nodes (like Rust's `struct_item` or Go's `type_identifier`) into standard `classes, functions,` and `imports` dictionaries.

3. Dependency Graphing & Centrality Analysis

With the AST extracted, the agent maps the actual execution flow of the software.

- **Regex Edge Building:** The `build_dependency_graph` tool cross-references extracted import statements against known file stems in the repository. It uses strict regex word boundaries (`\b`) to prevent false-positive matches (e.g., ensuring an import of "app" does not falsely trigger on "apple").
- **Graph Centrality (Critical Nodes):** The `analyze_graph` node mathematically analyzes the adjacency list. It calculates "Out-Degree" to identify the central orchestrator files (files that import many others) and "In-Degree" to identify core interfaces (files imported by many others).
- **Automated Code Reading:** The agent isolates these mathematically critical files, reads their first 150 lines to protect the LLM context limit, and generates dense, 3-sentence summaries of what those specific subsystems do.

4. Semantic RAG Engine

For targeted user queries, the system builds an embedded vector search space.

- **Chunking Strategy:** The `RecursiveCharacterTextSplitter` slices the validated code files into 1,000-character chunks with a 200-character overlap.
- **Embeddings & Storage:** These chunks are processed by the locally cached `all-MiniLM-L6-v2` HuggingFace embedding model and persisted into a Chroma database.

- **Retrieval:** The RAG engine pulls the top 5 most relevant code snippets, appending the exact file source metadata to each chunk before passing it to the ReAct planner.

5. Pedagogical Generation Engine

The final generation node does not output dry lists of files. It takes on the persona of a "Principal Software Engineer" giving a whiteboard breakdown.

- **Dynamic Instruction Logic:** If the user asks a specific question, the engine pivots to a "Deep-Dive to Summary" framework, instantly explaining the core mechanics of the retrieved RAG snippets before summarizing the architecture.
- **Data Camouflage:** Complex, nested JSON outputs from the AST parser are cleanly flattened into readable text block summaries, hiding the robotic data structures and feeding the LLM a natural narrative of dependencies.

IV. Hardware Acceleration & Telemetry

This project leverages cutting-edge hardware optimizations to run complex LLM reasoning locally on AMD architecture.

- **Inference Backend:** Connects to an AMD Radeon server running vLLM over port 8000.
- **Live GPU Monitoring:** A dedicated telemetry script scrapes the vLLM `/metrics` endpoint in real-time.
- **Hardware Metrics Tracked:** The monitor captures active processing requests, total prompt/generation tokens, and precise GPU KV Cache VRAM usage percentages.

V. Project Structure

1. `app.py` (The Streamlit Frontend)

This file serves as the interactive dashboard for the user. It initializes the Streamlit UI, manages session states with unique thread IDs, and provides a conversational chat interface. It actively streams the LangGraph execution steps to the screen, surfacing the Planner's inner strategy and any QA rejections directly to the user. It also features a reset mechanism that securely wipes the local vector database and temporary repository folders.

2. `graph.py` (The ReAct Orchestrator)

This is the brain of the agent. It defines the `AgentState` schema and registers all graph nodes, including the RAG engine, AST builder, and reflection loop. The graph is compiled into a cohesive state machine that controls the flow of data from initial repository cloning all the way to final markdown generation.

3. `main.py` (The Extraction Engine)

This module contains the deterministic Python logic. It features the `tree-sitter` language registry for cross-language parsing and handles the complex regex logic required to build the file-to-file dependency graph.

4. `monitor.py` (The Hardware Telemetry)

A lightweight diagnostic script that continuously polls the AMD Radeon cloud instance. It parses the Prometheus-style metrics dump to extract human-readable hardware statistics, proving hardware utilization.

VI. Installation & Setup

The project relies on a robust stack of modern AI libraries.

Prerequisites: Ensure you have Python installed, along with the dependencies listed in the provided configuration. Key libraries include:

- `langchain` and `langgraph` (Framework)
- `streamlit` (UI)
- `chromadb` and `sentence-transformers` (Vector Search)
- `tree-sitter` (AST Parsing)

Execution:

1. Clone this repository and install dependencies via `pip install -r requirements.txt`.
2. Configure your `RADEON_BASE_URL` and `RADEON_API_KEY` in `graph.py`.
   PY
3. Run the application: `streamlit run app.py`.
4. (Optional) In a separate terminal, run `python monitor.py` to watch live AMD GPU telemetry.

VII. Conclusion & Future Prospects

1. Summary & Impact

The **Autonomous Codebase Architect** demonstrates how combining static code analysis (`tree-sitter` ASTs and dependency graphing) with agentic LLM reasoning (`langgraph` ReAct loops) significantly lowers the barrier to understanding complex software systems. By offloading heavy compute tasks to dedicated AMD Radeon Cloud GPUs running quantized

vLLM inference, the system delivers near-instant architectural insights without overloading local developer hardware.

2. Future Prospects

To take this platform from a hackathon prototype to an enterprise-grade developer tool, the following enhancements are planned:

- **VS Code & IDE Extensions:** Package the agent into a native IDE sidebar extension, allowing engineers to query architecture and data flows directly from their active code editor.
- **CI/CD Pipeline Integration:** Automate architectural documentation updates via GitHub Actions, regenerating vector embeddings and onboarding guides on every merged pull request.
- **Cross-Repository Microservice Graphs:** Expand the dependency graph builder to parse multi-repo architectures (e.g., tracing API schemas across gRPC and REST interfaces between distinct microservices).
- **Interactive Visual Graph UI:** Integrate interactive 3D graph visualizations (using libraries like `pydeck` or `Three.js`) directly into the Streamlit UI, allowing developers to click on nodes to inspect AST metrics and triggering targeted RAG searches visually.
- **Multi-GPU Cluster Scaling:** Scale the vLLM backend across multi-GPU AMD clusters to support asynchronous processing of multi-million-line enterprise monorepos.